



TensorRT for Edge Inference

Accelerating deep learning on NVIDIA Jetson & GPUs

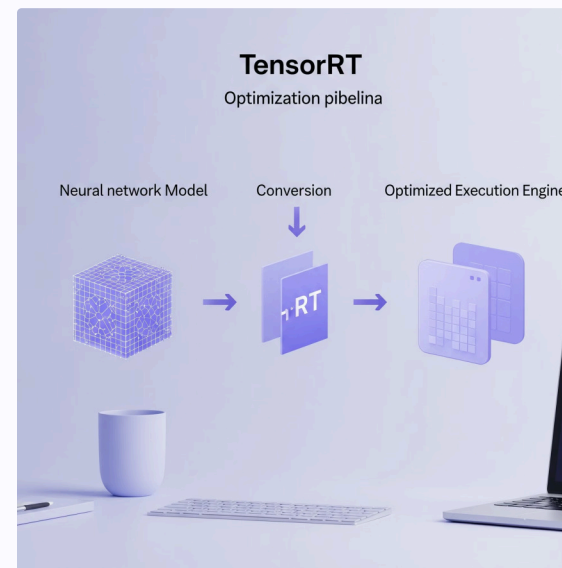
A technical overview for engineers and developers building edge AI solutions

What Is TensorRT?

NVIDIA's **inference optimizer & runtime** that converts trained models into optimized execution engines for deployment on edge devices.

Supported Targets

- Jetson Family (Nano, Xavier, Orin)
- NVIDIA GPUs (consumer & professional)
- Data center AI accelerators



Framework Support

Works with models from [PyTorch](#), [TensorFlow](#), and [ONNX](#) format



Why TensorRT for Edge AI?

Reduced Latency

Faster inference with optimized FP16/INT8 kernels for real-time applications

Smaller Memory Footprint

Compact model representation fits on resource-constrained edge devices

Increased Throughput

Better batch handling for processing multiple inputs simultaneously

Extended Battery Life

Energy-efficient execution critical for IoT devices and mobile robotics

Optimization Techniques in TensorRT

Layer Fusion

Combines multiple operations into fewer optimized kernels, reducing memory transfers

Precision Calibration

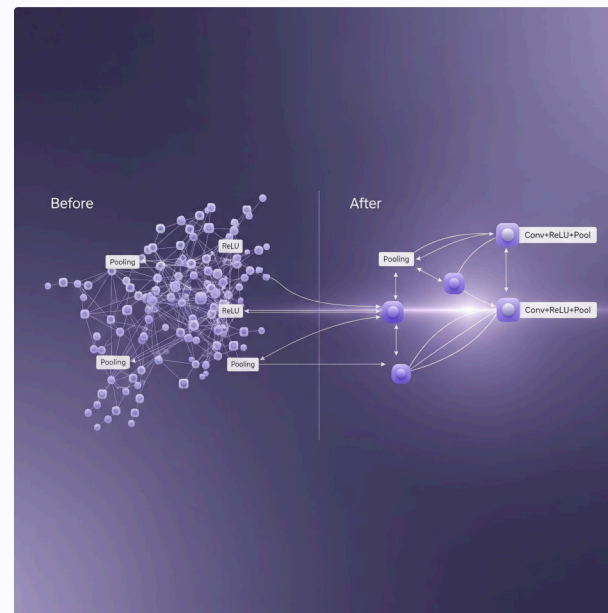
Intelligently converts operations from FP32 $\rightarrow$ FP16 $\rightarrow$ INT8 with minimal accuracy loss

Kernel Auto-Tuning

Selects optimal implementation for each operation based on target hardware capabilities

Dynamic Memory Allocation

Efficiently reuses tensor memory across the execution graph



TensorRT analyzes the computational graph and applies hardware-specific optimizations

TensorRT Workflow Overview

Train Model

Develop and train your neural network using PyTorch or TensorFlow

Export to ONNX

Convert your trained model to the Open Neural Network Exchange format

Build TensorRT Engine

Generate an optimized .engine file specifically for your target hardware

Run Inference

Deploy the engine with the TensorRT runtime in your application

Benchmark & Validate

Verify performance gains and ensure accuracy meets requirements

Export to ONNX (PyTorch)

ONNX provides a standardized intermediate representation that TensorRT can optimize

```
import torch
import torchvision.models as models

# Load a pretrained model
model = models.resnet18(pretrained=True).eval()

# Create dummy input with correct dimensions
dummy = torch.randn(1, 3, 224, 224)

# Export to ONNX format
torch.onnx.export(model,
                  dummy,
                  "resnet18.onnx",
                  input_names=["input"],
                  output_names=["output"])
```

This creates an ONNX file that preserves the model architecture and weights while being framework-independent

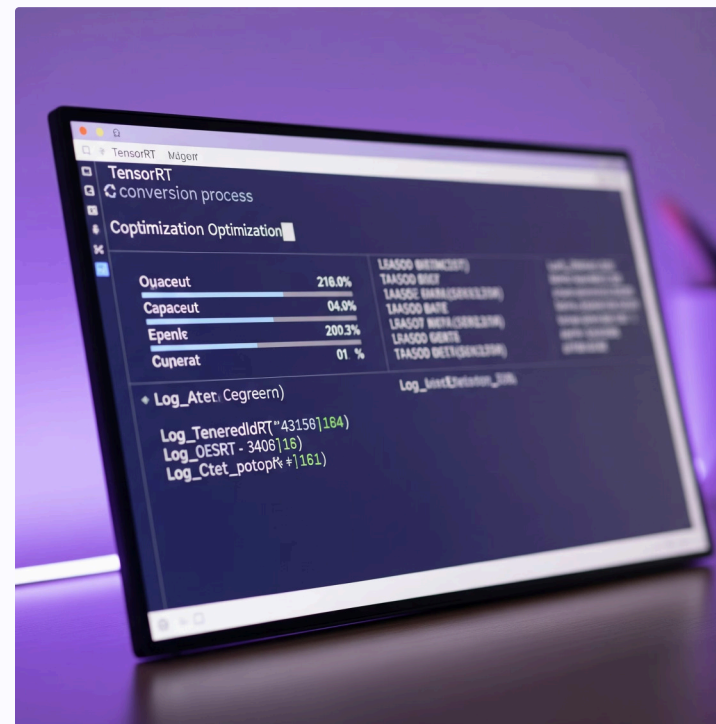
Convert ONNX → TensorRT Engine

Using trtexec Command-Line Tool

```
# FP16 optimization
trtexec --onnx=resnet18.onnx \
  --saveEngine=resnet18_fp16.engine \
  --fp16

# INT8 optimization with calibration
trtexec --onnx=resnet18.onnx \
  --saveEngine=resnet18_int8.engine \
  --int8
```

`trtexec` is bundled with TensorRT and provides a simple way to convert and benchmark models



The generated `.engine` file contains hardware-specific optimizations and is ready for deployment to your target device

Run Inference with TensorRT (Python)

```
import tensorrt as trt
import pycuda.driver as cuda
import pycuda.autoinit
import numpy as np

# Load the TensorRT engine
TRT_LOGGER = trt.Logger()
with open("resnet18_fp16.engine", "rb") as f:
    engine = trt.Runtime(TRT_LOGGER).deserialize_cuda_engine(f.read())

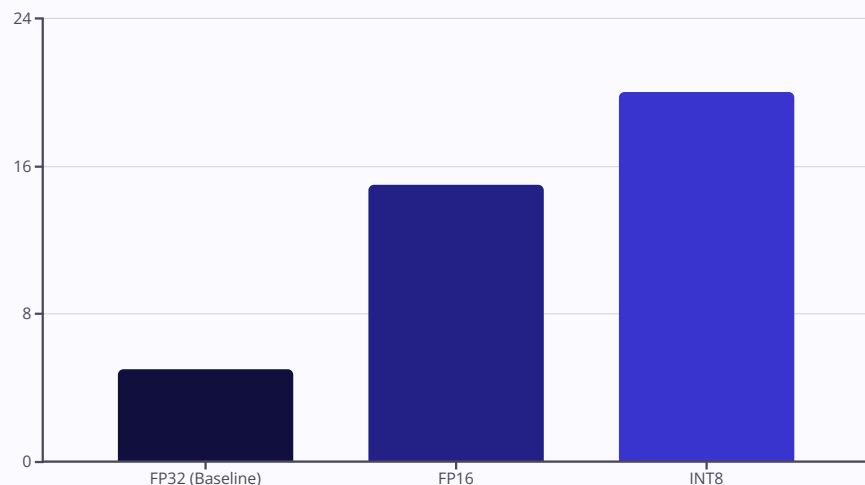
# Create execution context
context = engine.create_execution_context()

# Prepare input data
input_data = np.random.rand(1, 3, 224, 224).astype(np.float32)

# Additional code for input/output buffer allocation and inference execution
```

❏ Helper libraries like **Torch-TensorRT** can significantly simplify this process for PyTorch users, providing a more streamlined API.

Performance Gains with TensorRT



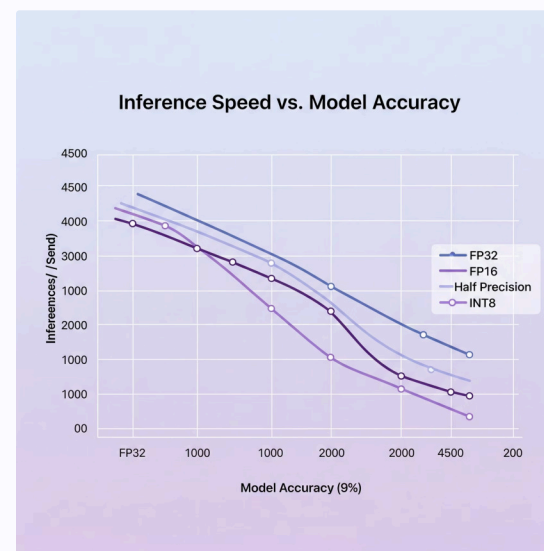
Typical performance on Jetson Nano. Xavier and Orin platforms achieve even greater gains (30-100+ FPS)

Performance vs. Accuracy Trade-off

INT8 quantization may result in ~1-2% accuracy loss compared to FP32

Mitigate with:

- Proper calibration datasets
- Quantization-Aware Training (QAT)
- Per-layer precision control



Use Cases at the Edge

Computer Vision

YOLO, SSD, and ResNet models for real-time video analytics, enabling object detection and classification at the edge

Robotics

SLAM, navigation, and perception algorithms running on battery-powered robots requiring low-latency decision making

Healthcare

Portable diagnostic imaging devices and patient monitoring systems that process data locally for privacy and speed

Retail/Smart Cities

Surveillance systems, people counting, and behavior analytics applications that need to process multiple video streams

TensorRT Best Practices

1

Representative Calibration Data

Use diverse, production-like datasets for INT8 calibration to ensure accuracy across all inputs

2

Profile Before Production

Benchmark latency and throughput with `trtexec` under various batch sizes and precision modes

3

Maintain Multiple Engines

Keep both FP16 and INT8 engines for fallback options when accuracy requirements change

4

Scale with Kubernetes + Triton

For multi-device deployments, use NVIDIA Triton Inference Server with Kubernetes orchestration

5

Validate Accuracy vs. Speed

Always quantify the trade-off between performance gains and accuracy loss for your specific application



⚠ Edge AI applications often have strict latency requirements. Always test under real-world conditions, including thermal constraints.

Key Takeaways

TensorRT is NVIDIA's edge inference accelerator

Purpose-built for maximizing AI performance on resource-constrained edge devices

Achieve major speedups with precision optimizations

FP16 and INT8 quantization can deliver 3-4x performance improvements on Jetson devices

Simple workflow: train → ONNX → TensorRT engine

Straightforward integration with existing PyTorch and TensorFlow development pipelines

Essential for real-time AI at the edge

Enables applications that were previously impossible on power-constrained devices

Get started at: developer.nvidia.com/tensorrt

